AI Chatbot for Student Query Assistance and Campus Information

A CAPSTONE PROJECT REPORT

Submitted in the partial fulfilment for the Course of

CSA1709 - ARTIFICIALINTELLIGENCE

to the award of the degree of

BACHELOR OF

ENGINEERING IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Pugazhendi K [192311026]

Under the Supervision of
Dr P Rajaram

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical

Sciences Chennai-602105

Sep 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

Engineer to Excel

DECLARATION

We, **Gondel Prashanth, Shaik Mahaboob Basha, Pugazhendi K** of the department of COMPUTER SCIENCE AND ENGINEERING Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled 'AI Chatbot for Student Query Assistance and Campus Information' is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 4-09-2026

Signature of the Students with Names

Pugazhendi K [192311026]

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

Engineer to Excel

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "AI Chatbot for Student Query Assistance and Campus Information" has been carried out by Pugazhendi K UNDER the supervision of Dr Guna G Dr. Mohammad Irshad and is submitted in partial fulfilment of the requirements for the current semester of the

B.E COMPUTER SCIENCE AND ENGINEERING program at Saveetha Institute of Medical and Technical Sciences, Chennai.

SIGNATURE
Dr.Rashmita Khilar
Program Director
Computer Science Of Engineering
Saveetha School of Engineering
SIMATS

SIGNATURE
Dr P Rajaram
Professor
Computer Science Of Engineering
Saveetha School of Engineering
SIMATS

Submitted for the Project work Viva-Voce held on 4-09-2026

.

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. Ashwani Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process.

We are especially indebted to our guide, Dr Rajaram P for his creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

Pugazhendi K [192311026]

ABSTRACT

Educational institutions receive a large number of repetitive student queries related to courses, examinations, academic schedules, events, campus facilities and administrative services. Providing timely and consistent answers to these queries through manual support requires continuous staff involvement and may lead to delays, especially outside working hours. This project proposes an AI Chatbot for Student Query Assistance and Campus Information to provide automated, real-time and accessible support through conversational interfaces.

The proposed system uses Natural Language Processing to understand student messages and identify the purpose of a query. Intent classification is used to categorize questions such as course information, examination schedules, events, fees, library services and campus facilities. Named Entity Recognition and entity extraction are used to identify important information such as course names, departments, dates and event names. A dialogue management component maintains the conversational flow, retrieves relevant information from a MySQL database and generates appropriate responses.

The system is designed with a Python and Flask backend and can use NLP libraries such as NLTK and spaCy together with chatbot frameworks such as Rasa or Dialogflow. A web-based chat interface developed using HTML, CSS and JavaScript provides an accessible user interaction channel. The architecture can also be extended to WhatsApp and cloud deployment for multi-channel access.

The proposed chatbot is evaluated using parameters including intent classification accuracy, response latency, entity extraction accuracy, conversation coverage and fallback rate. The project demonstrates how Artificial Intelligence and Natural Language Processing can be applied to improve student support services by reducing response time, increasing information accessibility and providing a scalable foundation for future smart-campus applications.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLE

CHAPTER 1
INTRODUCTION

1.1 Background Information

With the rapid growth of Artificial Intelligence, conversational systems are increasingly used to provide automated support in domains where users repeatedly request information. Educational institutions are one such environment. Students frequently need information about courses, examinations, timetables, events, fees, library services, hostel facilities, placements and other campus-related activities.

Traditional student support generally depends on office staff, faculty members, notice boards, websites and communication groups. Although these sources are useful, students may experience delays when information is scattered across multiple locations or when staff are unavailable. A conversational chatbot can provide a single point of interaction through which students ask questions using natural language.

The proposed project, AI Chatbot for Student Query Assistance and Campus Information, is designed to understand student queries, identify the intended requirement and provide relevant responses in real time. The system combines Natural Language Processing, intent classification, entity extraction, dialogue management and database retrieval.

For example, the student query "When is the Artificial Intelligence examination?" can be processed to identify the intent as Examination Information and the subject as Artificial Intelligence. The chatbot can then retrieve the corresponding information from the database and provide a suitable response.

The project is organized around three major modules: Natural Language Understanding and Intent Recognition, Dialogue Management and Response Generation, and Deployment and Multi-channel Integration. Together, these modules form a scalable architecture for a smart-campus support system.

1.2 Project Objectives

- Design and develop an AI-based chatbot for student query assistance.
- Understand student messages written in natural language.
- Classify queries into appropriate intents such as courses, examinations and events.
- Extract important entities such as course names, departments, dates and event names.
- Manage multi-turn conversations and maintain contextual information.
- Retrieve relevant campus information from a MySQL database.
- Generate accurate and understandable responses in real time.
- Provide fallback and human-support handoff for unresolved queries.
- Support deployment through web and extend the system to WhatsApp.

1.3 Significance of the Project

The project is significant because it demonstrates the practical application of Artificial Intelligence and Natural Language Processing in the education sector. Instead of requiring students to search through multiple sources, the chatbot provides a conversational interface for obtaining relevant information.

- Provides faster access to frequently requested information.
- Reduces repetitive workload for administrative staff.
- Improves availability of support beyond office hours.
- Offers consistent responses using a centralized knowledge source.
- Demonstrates practical NLP concepts such as intent classification and entity extraction.
- Supports scalable integration with web and messaging platforms.
- Creates a foundation for future smart-campus services.

1.4 Scope of the Project

The scope of the proposed AI Chatbot for Student Query Assistance and Campus Information is focused on providing an intelligent and accessible platform for students to obtain academic and campus-related information through natural language conversations. The chatbot is designed to handle common student queries related to courses, examination schedules, academic events, campus facilities, library services, hostel information, placements, and frequently asked questions. Instead of requiring students to search through multiple websites, notice boards, or administrative offices, the system provides a single conversational interface through which information can be accessed quickly.

1.5 Methodology Overview

The project follows a structured development process consisting of research and requirement analysis, dataset preparation, system design, implementation, testing and evaluation.

- Research and Analysis: Identify common student queries and define the required chatbot capabilities.
- Data Preparation: Create intents, sample utterances, entities, responses and structured campus information.
- System Design: Define the architecture connecting the chat interface, NLP engine, dialogue manager and database.
- Implementation: Develop the chatbot modules using Python, NLP libraries, Flask and MySQL.
- Testing: Test valid queries, ambiguous queries, database retrieval and fallback conditions.
- Evaluation: Measure intent accuracy, entity extraction performance, response latency, conversation coverage and fallback rate.

CHAPTER 2
PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem

Students require information from different departments and services throughout an academic semester. A student may need to know an examination date, course details, event schedule or campus service timing. In many cases, information is distributed across websites, notices and staff members. The student must therefore search multiple sources or wait for human assistance.

The main problem addressed by this project is the absence of a single intelligent conversational interface that can understand different forms of student questions and provide relevant information immediately. The problem is not only retrieval of information; the system must also understand natural language, determine the purpose of the query, identify important details and decide how to respond.

2.2 Evidence of the Problem

The need for an intelligent student query assistance system can be observed in educational institutions where students frequently require information from different departments and administrative services. Students often ask repeated questions regarding examination schedules, course details, event dates, library timings, hostel facilities, placement activities, and other campus services. Answering these questions manually requires continuous involvement from faculty members and administrative staff, which can increase their workload.

Another issue is that campus information is often distributed across different sources such as official websites, notice boards, student groups, departmental offices, and online portals. Students may need to search through several sources before finding the required information. This process can be time-consuming and may lead to confusion when information is not easily accessible or when multiple versions of information are available.

Traditional information systems also depend heavily on office timings and the availability of staff members. Students may require assistance outside normal working hours, especially during examination periods, assignment deadlines, or campus events. Static FAQ pages can provide some information, but they may not understand natural language questions or provide personalized clarification.

2.3 Stakeholders

Students: The primary users who receive instant support for academic and campus-related queries.

Faculty Members: Benefit from reduced repetition of routine informational questions.

Administrative Staff: Can focus on complex issues while routine queries are automated.

Institution Management: Can use the system as part of a digital and smart-campus strategy.

Technical Administrators: Maintain the knowledge base, database and chatbot services.

Human Support Team: Receives escalated conversations that cannot be resolved automatically.

2.4 Supporting Data and Research

The project is supported by concepts from Natural Language Processing, information retrieval and conversational Artificial Intelligence. A chatbot generally requires a representation of user intentions, examples of user utterances, relevant entities and suitable response mechanisms. For institutional use, structured information such as course data, examination schedules and event details can be maintained in a database and retrieved when required.

The proposed system combines the following approaches:

- Natural Language Processing for understanding text.
- Intent Classification for identifying the purpose of a query.
- Named Entity Recognition and entity extraction for identifying important information.
- Rule-based dialogue management for predictable conversational flows.
- Database retrieval for dynamic campus information.
- Fallback handling for unknown or low-confidence queries.
- Multi-channel integration through web and messaging APIs.

2.5 Problem Analysis Summary

The analysis shows that an effective student-support chatbot must combine language understanding with reliable information retrieval. A simple keyword-matching system is insufficient because students may ask the same question using different words. Therefore, the proposed solution identifies intent, extracts entities, manages the dialogue and retrieves information from a structured database. The system also requires a fallback mechanism because not every query can be handled automatically.

CHAPTER 3
SOLUTION DESIGN AND IMPLEMENTATION

3.1 Development and Design Process

The development and design process of the proposed AI chatbot follows a structured software engineering approach consisting of requirement analysis, system design, implementation, integration, testing, and evaluation. The first stage involves identifying the common types of queries that students may ask and defining the functional requirements of the chatbot. Based on these requirements, the system is divided into separate modules so that each component can perform a specific responsibility.

The design of the system focuses on three major modules. The first module, Natural Language Understanding and Intent Recognition, processes student messages and identifies their purpose. The second module, Dialogue Management and Response Generation, determines how the chatbot should respond based on the identified intent, extracted entities, and conversation context. The third module, Deployment and Multi-channel Integration, connects the chatbot with the web interface and provides the foundation for integration with other platforms.

A modular design approach is used to improve maintainability and scalability. Each module communicates with other components through clearly defined data flows and APIs. The chatbot interface sends the student message to the backend, where it is processed using NLP techniques. The result of the language analysis is passed to the dialogue management component, which retrieves relevant information from the database and generates an appropriate response.

The development process also includes testing different types of queries, including valid queries, incomplete queries, ambiguous queries, and unknown queries. Performance is evaluated using parameters such as intent classification accuracy, entity extraction accuracy, response latency, conversation coverage, and fallback rate. This systematic development process ensures that the chatbot can be improved and extended in future versions.

3.2 Tools and Technologies Used

NLTK and spaCy are used for Natural Language Processing tasks. NLTK can be used for operations such as tokenization, stop-word processing, and text preprocessing, while spaCy provides advanced language processing capabilities and Named Entity Recognition. These tools help the chatbot understand the structure and important information contained in student messages.

A chatbot framework such as Rasa or Dialogflow can be used for intent recognition and conversation management. These frameworks provide mechanisms for defining intents, training models, managing dialogue flows, and handling different conversational situations. The chatbot can also use rule-based response mechanisms for predictable queries and fallback handling.

Flask is used as the backend framework for developing APIs that connect the chat interface with the NLP and chatbot components. The backend receives user messages, processes them, communicates with the database, and returns responses to the frontend. MySQL is used as the database for storing structured information such as course details, examination schedules, events, facilities, FAQs, and conversation-related data.

The frontend is developed using HTML, CSS, and JavaScript, which provide an interactive web-based chat interface. REST APIs are used for communication between the frontend and backend. The system can be deployed on a cloud platform to make the chatbot accessible through the internet and can later be extended to integrate with WhatsApp or other communication platforms.

3.3 Solution Overview

The proposed solution operates as an intelligent conversational system that processes student queries and provides relevant responses based on Natural Language Processing and database information retrieval. The process begins when a student enters a message through the web chat interface or another supported communication channel. The message is sent to the Flask backend, where it is processed by the Natural Language Understanding module.

The system first performs text preprocessing to prepare the input for analysis. The chatbot then performs intent classification to determine the purpose of the student's query. For example, a query asking about an examination date can be classified as an examination-related intent. At the same time, the system extracts important entities from the query, such as the course name, department, date, or event name.

The identified intent and entities are then passed to the dialogue management module. This module determines the appropriate action based on the available information and the current conversation context. If the required information is available in the MySQL database, the system retrieves the relevant records and prepares a response.

3.4 Engineering Standards

The project follows recognized software engineering practices to improve maintainability, reliability and clarity.

- Structured Software Development Lifecycle: requirements, design, implementation, testing and evaluation.
- Modular Design: separation of NLP, dialogue management, database and user interface components.
- Coding Standards: readable naming, documentation and consistent code structure.
- API Design: clear request and response formats.
- Database Design: structured tables and controlled access.
- Testing: functional, integration and negative test cases.
- Maintainability: independent modules allow future changes without redesigning the entire system.
- Security Considerations: validation of user input and controlled database access.

3.5 Solution Justification

The proposed architecture is justified because it separates language understanding from response management and data storage. NLP techniques allow the chatbot to understand multiple ways of asking similar questions. Entity extraction provides important details required for accurate information retrieval. The dialogue manager controls the conversation and the database provides a structured source for campus information.

A Flask-based backend provides a lightweight way to connect the user interface with NLP services and databases. The web interface provides immediate accessibility, while the multi-channel architecture allows future integration with WhatsApp or other platforms. Together, these components provide a practical foundation for an intelligent student-support system.

CHAPTER 4
RESULTS AND RECOMMENDATIONS

4.1 Evaluation of Results

The proposed AI chatbot is evaluated based on its ability to understand student queries and provide appropriate responses. The primary performance parameters include intent classification accuracy, entity extraction accuracy, response latency, conversation coverage, and fallback rate. These parameters help determine whether the chatbot is able to correctly identify user requirements and respond within an acceptable time.

The system is expected to successfully handle common queries related to courses, examinations, events, campus facilities, and frequently asked questions. When a valid query is entered, the chatbot processes the message, identifies the intent, extracts relevant entities, and retrieves information from the available knowledge base or database.

The response latency is another important factor because students expect immediate interaction from a chatbot. A well-designed system should process queries and return responses quickly. The fallback mechanism is evaluated by testing unknown and ambiguous queries. Instead of generating incorrect information, the chatbot should ask for clarification or provide a suitable fallback response.

4.2 Challenges Encountered

Several challenges may be encountered during the development of the AI chatbot. One of the major challenges is preparing a sufficient dataset containing different ways students may express the same query. For example, students can ask about an examination schedule using many different sentence structures. The chatbot must recognize these variations as belonging to the same intent.

Another challenge is distinguishing between similar intents. Queries related to course information and examination information may contain similar course names but require different responses. Accurate intent classification therefore requires carefully designed training examples.

Entity extraction can also be challenging because course names, department names, dates, and event names may appear in different formats. Incomplete queries present another challenge. For example, if a student asks, “When is the exam?” without specifying the subject, the chatbot must request additional information.

4.3 Possible Improvements

The proposed chatbot can be improved in several ways in future versions. Advanced Transformer-based models can be introduced to improve the understanding of complex student queries. Multilingual support can also be added so that students can communicate with the chatbot in multiple languages.

Voice-based interaction can be implemented using speech recognition and text-to-speech technologies. This would make the system more accessible to students who prefer voice communication. Personalized responses can also be provided by integrating the chatbot with authorized student information systems. An administrative dashboard can be developed to allow authorized staff to update examination schedules, event information, courses, and other campus data. The system can also include analytics to identify frequently asked questions and areas where students require additional support.

The chatbot can further be integrated with Learning Management Systems, college ERP systems, mobile applications, and additional messaging platforms.

4.4 Recommendations

The chatbot should be tested using a large variety of realistic student queries before deployment. Training data should contain multiple variations of each query to improve intent classification. The knowledge base and database should also be updated regularly to ensure that students receive current information.

A confidence threshold should be used to prevent the chatbot from giving incorrect answers when it is uncertain about the user's intent. Queries that cannot be resolved automatically should be directed to human support.

Security and privacy should also be considered when integrating the system with student information. Authentication and controlled access should be used when personal information is involved.

Continuous monitoring and evaluation are recommended after deployment. Conversation logs can help identify new types of student queries and improve the chatbot over time.

CHAPTER 5
REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

5.1 Key Learning Outcomes

The development of this project provided valuable academic and practical learning experiences. The project required the application of Artificial Intelligence, Natural Language Processing, web development, database management, and software engineering concepts. The learning outcomes can be divided into academic knowledge, technical skills, and problem-solving and critical-thinking abilities.

5.1.1 Academic Knowledge

The project improved the understanding of theoretical concepts related to Artificial Intelligence and Natural Language Processing. Concepts such as intent classification, Named Entity Recognition, tokenization, text preprocessing, dialogue management, and response generation were studied and applied in a practical system.

The project also improved the understanding of how conversational AI systems work. Rather than treating a chatbot as a simple question-and-answer application, the project demonstrated the importance of language understanding, context management, data retrieval, and fallback mechanisms.

Knowledge related to software engineering was also strengthened. The project required requirement analysis, modular system design, implementation planning, testing, and evaluation. These concepts helped in developing a structured approach to solving the problem.

5.1.2 Technical Skills

The project improved several technical skills required for modern software and AI development. Python programming skills were improved through the implementation of backend and NLP-related components. Knowledge of NLP libraries such as NLTK and spaCy was developed through text processing and entity extraction activities.

The project also provided experience with Flask for backend development and REST API communication. Database management skills were improved through the use of MySQL for storing and retrieving structured campus information.

Frontend development skills were also developed using HTML, CSS, and JavaScript to create a user-friendly chat interface. Integration between frontend, backend, NLP services, and databases provided practical experience in full-stack system development.

5.1.3 Problem-Solving and Critical Thinking

The project improved problem-solving and critical-thinking abilities by requiring the analysis of real-world challenges related to student information services. The development process involved identifying the actual problem, analyzing possible causes, and designing an appropriate technical solution.

Critical thinking was required when deciding how the chatbot should handle ambiguous queries, missing information, similar intents, and unsupported questions. Rather than assuming that every query could be answered automatically, the system design included clarification questions and fallback mechanisms.

The project also encouraged systematic debugging and testing. Problems were divided into smaller components, such as intent recognition, entity extraction, database retrieval, and frontend communication. This approach helped identify the source of errors and develop suitable solutions.

5.2 Challenges Encountered and Overcome

5.2.1 Personal and Professional Growth

Working on the project improved personal confidence and professional responsibility. The development process required consistent learning because the project involved multiple technologies and concepts. Difficulties related to implementation, data preparation, integration, and testing required patience and continuous improvement.

The project also improved time-management skills because different tasks had to be planned and completed within the project schedule. Tasks such as research, system design, implementation, debugging, documentation, and testing required proper organization.

Professional growth was achieved through the practice of structured development methods and documentation. The project demonstrated the importance of writing maintainable code, testing the system, documenting the architecture, and presenting technical work clearly.

5.2.2 Collaboration and Communication

Collaboration and communication played an important role in the successful development of the project. A capstone project involves dividing responsibilities, discussing technical decisions, sharing progress, and solving problems as a team.

Team members can contribute to different areas such as dataset preparation, NLP development, backend development, database design, frontend development, testing, and documentation. Effective

communication helps ensure that these components work together correctly.

5.3 Application of Engineering Standards

Engineering principles were applied through requirement analysis, modular architecture, structured implementation, systematic testing and documentation. The separation of language understanding, dialogue management, backend APIs and data storage improves maintainability. Test cases are used to verify both normal and failure conditions.

5.4 Insights into the Industry

The project provides insight into the growing use of conversational AI in customer service, education, healthcare, banking and enterprise systems. Real-world chatbot development requires more than an intent classifier. Reliable systems require data quality, integration with trusted information sources, security, monitoring, performance evaluation and a mechanism for human escalation.

5.5 Conclusion of Personal Development

The project improved technical knowledge and practical problem-solving ability. It strengthened understanding of Artificial Intelligence, NLP, web development, APIs and databases. The work also improved the ability to analyze a problem, divide it into modules, implement a solution, test different scenarios and identify future improvements. Overall, the project helped connect theoretical engineering concepts with a practical smart-campus application.

CHAPTER 6
CONCLUSION

The project “AI Chatbot for Student Query Assistance and Campus Information” presents an intelligent approach for improving access to academic and campus-related information. The system is designed to receive student queries in natural language, identify the intent, extract important entities and provide relevant responses through a conversational interface.

The proposed architecture combines Natural Language Processing, intent classification, entity extraction, dialogue management, database retrieval and web deployment. The three-module design separates language understanding, conversation management and multi-channel deployment, making the system easier to develop and extend.

The chatbot can reduce repetitive support workload and improve information accessibility by providing immediate responses for supported queries. Database integration enables dynamic information to be maintained independently from the conversational interface. Fallback handling and human-support escalation improve reliability when the system cannot confidently resolve a query.

The project provides a foundation for future smart-campus systems. Future work can include multilingual support, Transformer-based models, voice interaction, secure personalization, ERP integration, analytics and additional communication channels. In conclusion, the project demonstrates how Artificial Intelligence and Natural Language Processing can be integrated to develop a practical and scalable solution for student query assistance and campus information.

REFERENCES

1. Jurafsky, D., & Martin, J. H. Speech and Language Processing. Pearson / online manuscript.

2. Bird, S., Klein, E., & Loper, E. Natural Language Processing with Python. O'Reilly Media.

3. spaCy Documentation. Industrial-strength Natural Language Processing in Python.

4. NLTK Documentation. Natural Language Toolkit for Python.

5. Rasa Documentation. Open-source conversational AI framework.

6. Flask Documentation. Python web framework documentation.

7. MySQL Documentation. MySQL database reference and developer documentation.

8. Vaswani, A., et al. Attention Is All You Need. Advances in Neural Information Processing Systems.

9. Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.

10. Relevant institutional academic calendars, examination schedules, course information and event records used as the authorized knowledge source during implementation.

Appendices

Appendix A: System Architecture Diagram

The system architecture of the AI Chatbot for Student Query Assistance and Campus Information consists of multiple interconnected components that work together to understand student queries and provide relevant information. The process begins when a student enters a query through the web-based chat interface or another supported communication channel. The user query is sent to the Flask backend through an API request.

The backend forwards the query to the Natural Language Processing module, where the text is preprocessed and analyzed. Intent classification is then performed to identify the purpose of the student's query, such as course information, examination schedules, events, library services, or campus facilities. Entity extraction identifies important information contained in the query, including course names, department names, dates, and event names.

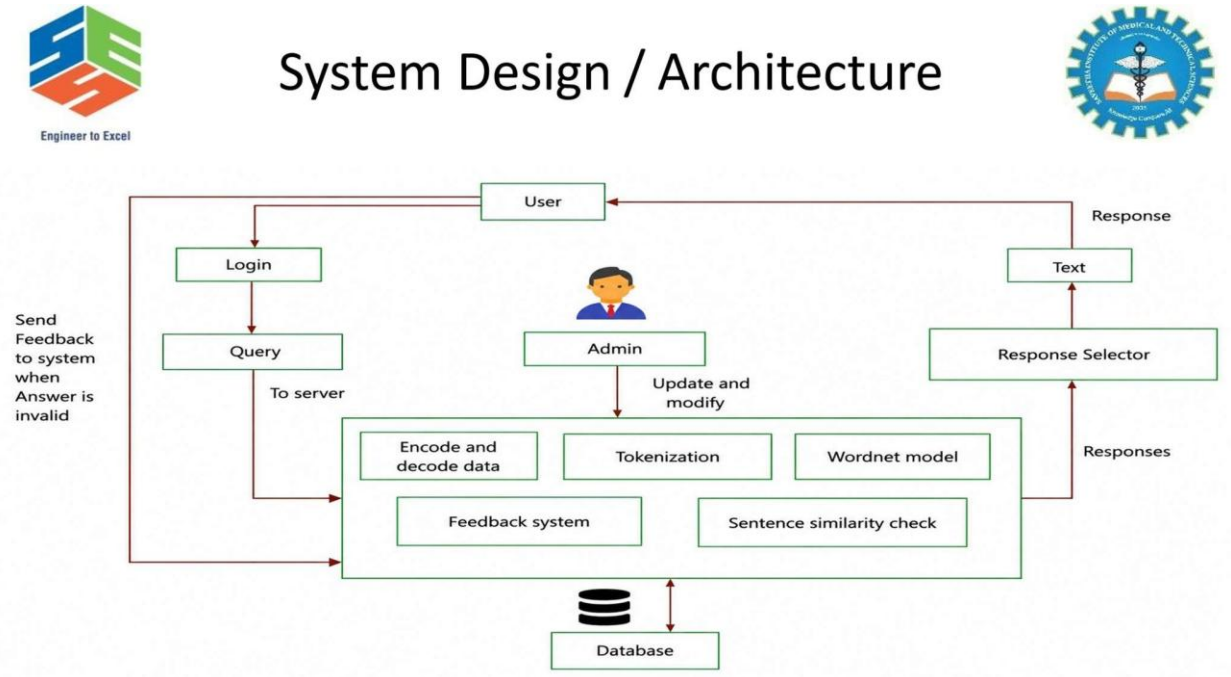

Figure A1: System Artitecture

Appendix B: Component List and Specifications

Component	**Technology / Specification**	Function
User Interface	HTML, CSS, JavaScript	Provides the chat interface for students.
Backend Server	Python and Flask	Handles requests and connects all system modules.
NLP Module	NLTK and spaCy	Processes and analyzes student queries.
Database	MySQL	Stores campus and academic information.
API Communication	REST API	Enables communication between frontend and backend.
Cloud Deployment	Cloud Hosting Platform	Makes the chatbot accessible online.

Table A1: Components Used in the System

Appendix C: Control Logic and Working Algorithm

- Start the chatbot system and initialize the Flask backend, NLP components, and database connection.
- Receive the student query through the web chat interface.
- Validate the input to ensure that the student message is not empty or invalid.
- Preprocess the text by performing operations such as tokenization, normalization, and removal of unnecessary characters.
- Perform intent classification to identify the purpose of the student query.
- Extract important entities such as course names, departments, examination dates, and event names.
- Check the confidence level of the identified intent.
- If the confidence level is sufficiently high, send the intent and entities to the dialogue management module.
- Analyze the conversation context to determine whether the query is related to a previous message.

Appendix D: Testing and Observation Results

This appendix documents the system testing carried out under different simulated disaster conditions.

Test Scenario	Student Query / Input	Expected Result	Observation	Status
Greeting	Hello	Chatbot responds with a greeting	Correct greeting response generated	Pass
Course Query	Tell me about Artificial Intelligence	Course information displayed	Intent identified correctly	Pass
Examination Query	When is the AI exam?	Examination information retrieved	Relevant response generated	Pass
Event Query	What events are happening this week?	Event information displayed	Event intent processed	Pass

Table A2: System Testing and Validation Results

Appendix E: Response Efficiency Observation Summary

The response efficiency of the proposed AI chatbot is evaluated based on the ability of the system to understand student queries and generate appropriate responses within an acceptable time. The major factors considered during evaluation include intent classification accuracy, entity extraction accuracy, response latency, conversation coverage, and fallback rate.

During normal operation, queries that belong to predefined intents can be processed efficiently because the system follows a structured flow consisting of text preprocessing, intent identification, entity extraction, dialogue management, database retrieval, and response generation. The modular architecture helps reduce unnecessary processing and allows the chatbot to provide responses in real time.